

מודול 4 — סקריפטינג

חומר לימוד

1 יחידות

1. מודול 4 — סקריפטינג: Bash ו-Python (Scripting for Hackers)

מודול 4 — סקריפטינג: Python (Scripting for Bash ו-Hackers)

מטרות המודול: לרכוש את יסודות Python הנחוצים לאבטחת מידע — משתנים, מבני נתונים, לולאות ופונקציות — ולהגיע לכתיבת כלי אמיתי: **סורק פורטים (Port Scanner)**.

למה Python? זו השפה הנפוצה ביותר בכתיבת כלי אבטחה ואוטומציה. רוב ה-Exploits, הסקריפטים והכלים בתחום כתובים בה. אין צורך להיות מתכנת — מספיק לדעת לקרוא, לשנות ולכתוב סקריפטים פשוטים.

3.1 יסודות: משתנים וטיפוסים (Variables & Types)

משתנה (Variable) הוא שם ששומר ערך.

```
target = "192.168.1.10" # מחרוזת (String)
port = 80                # מספר שלם (Integer)
is_open = True           # ערך בוליאני (Boolean)
timeout = 1.5            # מספר עשרוני (Float)
```

הטיפוסים הבסיסיים: **String** — טקסט, בין מרכאות: **"nmap"** - **Integer** — מספר שלם: **443** - **Float** — מספר עשרוני: **2.5** - **Boolean** — **True / False**.

3.2 מחרוזות (Strings)

מחרוזות הן טקסט, ויש עליהן פעולות שימושיות מאוד לעיבוד פלט של כלים:

```
banner = "Apache/2.4.49"
print(banner.upper())      # APACHE/2.4.49
print(banner.lower())      # apache/2.4.49
print(banner.split("/"))   # ['Apache', '2.4.49'] ← פיצול לרשימה
print("Apache" in banner)  # True ← בדיקת הכלה
print(banner.replace("2.4.49", "X")) # Apache/X
```

f-strings — הדרך הנוחה לשלב משתנים בטקסט:

```
ip = "10.0.0.5"
port = 22
print(f"Scanning {ip} on port {port}")    # Scanning 10.0.0.5 on port 22
```

3.3 מבני נתונים (Data Structures)

רשימה (List) — אוסף מסודר וניתן לשינוי

```
ports = [21, 22, 80, 443]
ports.append(8080)    # הוספה בסוף
print(ports[0])        # 21    ← איבר ראשון (אינדקס מתחיל מ-0)
print(len(ports))      # 5      ← אורך הרשימה
```

מילון (Dictionary) — זוגות מפתח:ערך

```
services = {80: "HTTP", 443: "HTTPS", 22: "SSH"}
print(services[80])    # HTTP
```

Tuple — כמו רשימה, אך אינו ניתן לשינוי

```
address = ("192.168.1.1", 80)    # IP + port יחד
```

מבנה	תחביר	ניתן לשינוי?	שימוש טיפוסי
List	[]	כן	רשימת פורטים/יעדים
Dictionary	{key: value}	כן	מיפוי פורט ← שירות
Tuple	()	לא	זוג IP+port קבוע

3.4 תנאים (Conditional Statements)

```
port = 22
if port == 22:
    print("SSH")
elif port == 80:
    print("HTTP")
else:
    print("Unknown service")
```

אופרטורים בוליאניים: == (שווה), != (שונה), <, >, <=, >= (שונה), and, or, not.

3.5 לולאות (Loops)

הלולאה היא הבסיס לאוטומציה — לסרוק טווח פורטים, לעבור על רשימת יעדים וכו'.

```
# על רשימה for לולאת
for port in [21, 22, 80, 443]:
    print(f"Checking port {port}")

# לולאה על טווח מספרים (1 עד 100)
for port in range(1, 101):
    print(port)

# לולאת while
count = 0
while count < 3:
    print(count)
    count += 1
```

3.6 פונקציות (Functions)

פונקציה אורזת קטע קוד לשימוש חוזר:

```
def scan_port(ip, port):
    print(f"Scanning {ip}:{port}")
    return True

scan_port("10.0.0.1", 80) # קריאה לפונקציה
```

3.7 מודולים (Importing Modules)

Python מגיע עם ספריות מובנות. שתי החשובות לרשתות:

```
import socket    # תקשורת רשת – לב הסודק
import sys       # אינטראקציה עם המערכת וארגומנטים
```

- **socket** — יוצר חיבורי רשת (בדיוק כמו שדפדפן מתחבר לשרת).

- ניתן להתקין ספריות נוספות עם `pip install <שם>`.

3.8 פרויקט: סורק פורטים (Building a Port Scanner)

זהו כלי אמיתי — הוא מנסה להתחבר לכל פורט בטווח, ומדווח אילו פתוחים. הוא מיישם את **שכבת התעבורה (TCP)** ואת **לחיצת היד** ממודול 2.

```
#!/usr/bin/env python3
import socket
import sys

# --- קלט מהמשתמש ---
target = "127.0.0.1" # היעד לסריקה (מעבדה בלבד!)

print(f"[*] Scanning target {target}")

try:
    # מעבר על הפורטים הנפוצים 1 עד 1024
    for port in range(1, 1025):
        # socket חדש (IPv4, TCP)
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        s.settimeout(0.5) # אל תחכה יותר מחצי שנייה

        # connect_ex אם החיבור הצליח (הפורט פתוח)
        result = s.connect_ex((target, port))
        if result == 0:
            print(f"[+] Port {port} is OPEN")
            s.close()

except KeyboardInterrupt:
    print("\n[!] Scan stopped by user")
    sys.exit()
except socket.gaierror:
    print("[!] Hostname could not be resolved")
    sys.exit()
```

כיצד זה עובד — שורה אחר שורה

1. `socket.socket(AF_INET, SOCK_STREAM)` — יוצר חיבור TCP מעל IPv4.
2. `settimeout(0.5)` — מונע מהסורק להיתקע על פורט חסום.
3. `connect_ex((target, port))` — מנסה את לחיצת היד; מחזיר 0 אם הפורט **פתוח**.
4. הלולאה חוזרת על כל פורט בטווח 1–1024.
5. בלוקי `try/except` מטפלים בעצירה ידנית (Ctrl+C) ובשגיאות DNS.

⚠ **הרץ אך ורק על 127.0.0.1 (המחשב שלך) או על מכונות במעבדה שלך.** סריקת פורטים של מערכת זרה ללא רשות עלולה להיחשב עבירה.

סיכום המודול

- **משתנים וטיפוסים** (String, Integer, Float, Boolean) הם אבני היסוד.
- **מבני נתונים**: List (מסודר, משתנה), Dictionary (מפתח:ערך), Tuple (קבוע).
- **תנאים ולולאות** מאפשרים החלטות ואוטומציה — הבסיס לכל כלי.
- **פונקציות** אורזות קוד לשימוש חוזר; **מודולים** כמו `socket` נותנים יכולות רשת.
- הרכבנו **סורק פורטים** אמיתי המיישם TCP ולחיצת יד ממודול 2.

תרגול מומלץ

1. הרץ את סורק הפורטים על `127.0.0.1` וזהה אילו פורטים פתוחים במחשבך.
2. שנה את הטווח לסרוק רק פורטים 20–100 והוסף הדפסת סיכום בסוף.
3. שדרג: הוסף שימוש ב- `services` (מילון) כדי להדפיס גם את שם השירות לצד כל פורט פתוח.